Serial Python Minimalis:
Tutorial #08-Larik 1D dengan NumPy

Ahmad R. T. Nugraha

ver. 21 April 2026

Kita telah mengetahui bahwa `list` pada Python merupakan urutan item yang dapat diindeks, diiterasi, ditambahkan elemen baru ke dalamnya (*append*), dan lain sebagainya:

```
lst = [1, 'a', 'pi', -0.25]
lst[2]
```

Output:

```
'pi'
```

```
lst.append(99.)
for item in lst:
    print(item)
```

Output:

```
1
a
pi
-0.25
99.0
```

Penggunaan `list` memang praktis untuk menyimpan sejumlah kecil data yang heterogen, tetapi kurang cocok untuk komputasi numerik berskala besar. Kreasi dan iterasi `list` bawaan Python bisa jadi lambat dan merepotkan. Sebagai contoh, misalkan kita ingin menghitung fungsi $y = \sin(x)$ untuk sebuah `list` berisi 10 nilai x yang berjarak sama dari 0 hingga 2π:

```
import math

xvals, yvals = [], []
n = 10
dx = 2 * math.pi / (n - 1)
```

```
for i in range(n):
    x = i * dx
    xvals.append(x)
    yvals.append(math.sin(x))
```

```
xvals
```

Output:

```
[0.0, 0.6981317007977318, 1.3962634015954636, 2.0943951023931953,
↪ 2.792526803190927, 3.490658503988659, 4.1887902047863905,
↪ 4.886921905584122, 5.585053606381854, 6.283185307179586]
```

```
yvals
```

Output:

```
[0.0, 0.6427876096865393, 0.984807753012208, 0.8660254037844388,
↪ 0.3420201433256689, -0.34202014332566866, -0.8660254037844384,
↪ -0.9848077530122081, -0.6427876096865396, -2.4492935982947064e-16]
```

Proses di atas cukup merepotkan. Kita harus menghitung jarak spasi antarnilai x, yaitu $\Delta x = \frac{2\pi}{n-1}$ untuk n titik, lalu perlahan-lahan mengiterasinya menggunakan sebuah pencacah (*counter*), i, yang bergerak dari 0 hingga $n-1$ guna membangun `list` `xvals` dan `yvals` satu demi satu menggunakan metode `append`.

Pustaka *Numerical Python* (NumPy) menyediakan cara yang jauh lebih efisien untuk mencapai hasil yang sama tanpa memerlukan perulangan (*loops*) secara eksplisit.

```
import numpy as np

x = np.linspace(0, 2 * np.pi, 10)
y = np.sin(x)
```

```
x
```

Output:

```
array([0.        , 0.6981317 , 1.3962634 , 2.0943951 , 2.7925268 ,
       3.4906585 , 4.1887902 , 4.88692191, 5.58505361, 6.28318531])
```

```
y
```

Output:

```
array([ 0.00000000e+00,  6.42787610e-01,  9.84807753e-01,  8.66025404e-01,
        3.42020143e-01, -3.42020143e-01, -8.66025404e-01, -9.84807753e-01,
       -6.42787610e-01, -2.44929360e-16])
```

Lebih jauh lagi, aplikasi aljabar linear secara umum sangat bergantung pada penggunaan entitas yang bersifat homogen dan berukuran tetap, seperti matriks matematika. Pada Python standar, matriks sebenarnya dapat direpresentasikan menggunakan `list` di dalam `list` (*lists of lists*). Akan tetapi, pendekatan ini tergolong kaku dan merepotkan karena bentuk sintaksisnya jauh berbeda dari notasi matematika yang selama ini kita kenal. Dalam konteks inilah, larik NumPy (NumPy *arrays*) menjadi struktur data andalan untuk komputasi numerik.

Sebelum membahas kasus multidimensi (khususnya dua dimensi), kita akan mempelajari larik (*array*) NumPy satu dimensi (1D) terlebih dahulu. Larik 1D ini dapat digunakan sebagai pengganti langsung untuk `list` pada Python. Ke depannya, kita tidak akan terus-menerus menyebut wadah data baru ini dengan nama panjang "NumPy *array*". Kita akan menyederhanakannya menjadi "larik" atau "*array*" saja, sama halnya seperti `list` bawaan inti Python cukup disebut "*list*".

1 Pembuatan Larik (*Array*) pada NumPy

Kita mulai dengan cara yang biasa, yaitu menggunakan `list` Python:

```
L = [11, 7, 19, 22, 55]
```

Langkah selanjutnya adalah mengimpor fungsionalitas NumPy yang dapat dilakukan dengan berbagai cara. Di sini kita menggunakan konvensi standar, yaitu mengimpor NumPy dengan nama singkatan:

```
import numpy as np
```

Cara termudah untuk membuat larik adalah dengan menggunakan fungsi `array()`, yang menerima entitas berbentuk urutan (*sequence-like entity*) dan mengembalikan sebuah larik yang berisi data yang diteruskan itu. Pada contoh kita, dengan menggunakan `list` `L`, kita mendapatkan:

```
xs = np.array(L)
xs
```

Output:

```
array([11,  7, 19, 22, 55])
```

Perhatikan bahwa fungsi `array()` merupakan bagian dari NumPy, sehingga kita harus menuliskan `np.array()` untuk mengaksesnya. Contoh ini dengan jelas menunjukkan bahwa *interpreter* Python memahami bahwa `xs` bukanlah sebuah `list`, melainkan sebuah objek larik. Perlu dicatat bahwa di sini terdapat fungsi `array()` dan objek larik yang saling berkaitan, dengan kata lain, fungsi `array()` itulah yang menciptakan objeknya. Perlu ditekankan juga bahwa ketika kita mencetak (*print*) `list` dan larik, outputnya akan terlihat sedikit berbeda:

```
print(xs)
print(L)
```

Output:

```
[11  7 19 22 55]
[11, 7, 19, 22, 55]
```

Tampilan larik dari NumPy di atas dapat kita lihat tidak menunjukkan tanda koma sehingga sangat membantu ketika kita perlu berfokus pada angka-angkanya itu sendiri, yang memang menjadi tujuan utama dirancangnya larik. Jika kita ingin melihat berapa banyak elemen yang terdapat di dalam larik `xs`, kita mungkin tergoda untuk mengambil fungsi bawaan inti Python `len()`. Namun, teknik ini dapat menyebabkan masalah di kemudian hari, sehingga jauh lebih baik menggunakan atribut `size` dari array `xs`:

```
xs.size
```

Output:

```
5
```

Ingat bahwa larik NumPy memiliki panjang yang tetap (*fixed-length*), sehingga jumlah total elemennya tidak dapat berubah. Dengan kata lain, tidak ada metode `append()` untuk larik sehingga ukurannya akan selalu tetap. Perilaku ini berbeda dengan `list`, yang jika dipanggil oleh `len(L)`, akan mengembalikan nilai yang berbeda sebelum dan sesudah kita menambahkan/menghapus elemen.

Atribut lain yang sangat berguna yang dimiliki larik NumPy adalah `dtype`, yakni tipe data dari elemen-elemen di dalamnya:

```
xs.dtype
```

Output:

```
dtype('int64')
```

Kita dapat langsung melihat bahwa meskipun kita tidak mendefinisikan tipe data secara eksplisit, elemen-elemen lariknya dapat menyimpulkan sendiri harus bertipe data apa. Ingat setiap larik NumPy bersifat homogen, yang berarti semua elemennya wajib memiliki tipe yang sama. Kita dapat melihat tipe yang dikembalikan bukanlah integer Python standar, melainkan `int64`. Angka ini memberi tahu kita berapa banyak bit yang digunakan untuk menyimpan integer tersebut dalam representasi mesin yang spesifik (64 bit membentuk apa yang oleh sebagian besar bahasa pemrograman disebut sebagai "*long integer*"). Kita juga dapat menggunakan `type()` bawaan Python untuk memeriksa tipe-tipe yang berbeda ini:

```
type(L)
```

Output:

```
list
```

```
type(L[0])
```

Output:

```
int
```

```
type(xs)
```

Output:

```
numpy.ndarray
```

```
type(xs[0])
```

Output:

```
numpy.int64
```

Output ini dengan jelas menunjukkan bahwa kita sedang berhadapan dengan tipe-tipe data yang berbeda. Tipe data NumPy dipetakan secara langsung ke representasi mesin.

Kita juga boleh memiliki larik yang berisi bilangan pecahan (*floats*):

```
ys = np.array([1.1, 2.2, 3.3, 4.4])
ys.dtype
```

Output:

```
dtype('float64')
```

Di sini kita menempatkan `list` yang digunakan untuk membuat array secara langsung sebagai argumen pada fungsi `array()`. Angka 64 di atas memberi tahu kita bahwa kita sedang berhadapan dengan bilangan pecahan presisi ganda (*double-precision float*). Pada contoh di atas, yakni `xs` dan `ys`, kita membiarkan NumPy menyimpulkan sendiri tipe argumennya. Namun, kita juga dapat memberikan tipe data tersebut sebagai argumen (opsional) eksplisit ke dalam fungsi `array()`:

```
zs = np.array([5, 6, 7, 8], dtype=np.float32)
zs
zs.dtype
```

Output:

```
dtype('float32')
```

Kita sengaja menggunakan `list` berisi bilangan bulat (integer) dan meminta NumPy untuk mempromosikannya menjadi bilangan pecahan (presisi tunggal). Dengan demikian, NumPy memberi kita kendali penuh (*fine-grained control*) atas representasi mesin mana yang ingin kita gunakan, khususnya untuk *floats*, yang dapat memiliki konsekuensi praktis yang besar dan merupakan salah satu nilai jual utama dari larik NumPy.

Sejauh ini, kita baru melihat cara membuat larik NumPy menggunakan bantuan `list` biasa (sebagai alat sementara). Sebagai contoh, jika kita menginginkan sebuah larik yang berisi sejumlah angka nol, mungkin kita akan melakukan sesuatu seperti:

```
np.array([0.]*5)
```

Output:

```
array([0., 0., 0., 0., 0.])
```

sedangkan untuk menghasilkan larik yang berisi sejumlah angka satu kita mungkin akan menuliskan:

```
np.array([1.]*4)
```

Output:

```
array([1., 1., 1., 1.])
```

Kasus-kasus penggunaan ini sangatlah umum sehingga NumPy memiliki fungsi bawaan untuk menghasilkan larik semacam itu secara langsung:

```
np.zeros(5)
```

Output:

```
array([0., 0., 0., 0., 0.])
```

```
np.ones(4)
```

Output:

```
array([1., 1., 1., 1.])
```

Seperti yang bisa kita lihat, kita meneruskan jumlah total elemen sebagai argumen pada setiap pemanggilan.

Fungsi NumPy lain yang sangat berguna untuk menghasilkan larik adalah `arange()`, yang merupakan generalisasi dari fungsi `range()` bawaan inti Python:

```
np.arange(10,16)
```

Output:

```
array([10, 11, 12, 13, 14, 15])
```

Secara alamiah kita dapat memberikan langkah (*stride*) sebagai argumen ketiga pada `arange()`, sama seperti yang kita lakukan pada `range()`. Pada titik ini, mereka yang baru mengenal NumPy sering kali baru menyadari bahwa `arange()` ternyata dapat menerima *floats* sebagai argumen:

```
np.arange(10.,16.)
```

Output:

```
array([10., 11., 12., 13., 14., 15.])
```

Namun, kita akan segera lihat bahwa teknik ini dapat mengundang masalah. Berikut adalah dua contoh yang menunjukkan kemungkinan kekacauan yang dihasilkan.

```
np.arange(1.5,1.75,0.05)
```

Output:

```
array([1.5 , 1.55, 1.6 , 1.65, 1.7 ])
```

```
np.arange(1.5,1.8,0.05)
```

Output:

```
array([1.5 , 1.55, 1.6 , 1.65, 1.7 , 1.75, 1.8 ])
```

Ingat kembali bahwa bilangan pecahan (dan operasi *floating-point*) tidak disimpan (atau dieksekusi) menggunakan presisi tak terbatas. Akibatnya, pada kasus pertama nilai akhirnya tidak diikutsertakan, sedangkan pada kasus kedua nilai akhirnya malah ikut masuk. Perbedaan ini bisa berdampak signifikan seperti pada kasus integrasi numerik. Saat melakukan perhitungan dengan rumus integral numerik tertentu, sangat penting agar menerapkannya secara tepat, misalnya menggunakan metode tertutup (yaitu, mengikutsertakan kedua titik akhir). Dokumentasi NumPy sangat eksplisit mengenai hal ini, "...apabila menggunakan langkah non-integer, seperti 0.1, hasilnya sering kali tidak konsisten...". Kita mungkin tergoda juga untuk menyelesaikan masalah ini dengan solusi akal-akalan seperti:

```
np.arange(1.5,1.75001,0.05)
```

Output:

```
array([1.5 , 1.55, 1.6 , 1.65, 1.7 , 1.75])
```

Namun, cara ini lagi-lagi adalah ide yang buruk. Sebagai gantinya, jauh lebih disarankan untuk menggunakan fungsi `linspace()` dari NumPy, yang mengembalikan sejumlah elemen dengan jarak yang persis sama sepanjang interval yang ditentukan:

```
np.linspace(1.5,1.75,6)
```

Output:

```
array([1.5 , 1.55, 1.6 , 1.65, 1.7 , 1.75])
```

```
np.linspace(1.5,1.8,7)
```

Output:

```
array([1.5 , 1.55, 1.6 , 1.65, 1.7 , 1.75, 1.8 ])
```

Perhatikan contoh di atas memberikan perilaku yang konsisten, tetapi dengan kompensasi bahwa kita harus memberikan sendiri jumlah elemen yang diinginkan. Kita mungkin bertanya-tanya mengapa fungsi `linspace()` diberi nama demikian. Jawabannya akan menjadi jelas setelah ditunjukkan bahwa NumPy memiliki fungsi yang disebut `logspace()`. Fungsi ini menghasilkan kisi-kisi titik (*grid of points*) dengan jarak logaritmik (perilaku bawaannya adalah menggunakan basis 10). Misalnya, jika kita menginginkan 5 titik yang didistribusikan secara logaritmik dari 10^{-1} hingga 10^{-5}, kita akan menuliskan:

```
np.logspace(-1, -5, 5)
```

Output:

```
array([1.e-01, 1.e-02, 1.e-03, 1.e-04, 1.e-05])
```

Argumen pertama adalah pangkat dari nilai awal, argumen kedua adalah pangkat dari nilai akhir, dan argumen ketiga adalah jumlah sampel yang akan dihasilkan. Tentunya jelas, rentang titik yang dihasilkan oleh `linspace()` bersifat linier. Ingat: baik `linspace()` maupun `logspace()` sama-sama menyertakan kedua titik akhirnya.

Dua perbedaan antara `list` dan larik yang kita temui sejauh ini (yakni, larik memiliki ukuran tetap dan bersifat homogen) mungkin tampak kurang begitu penting bagi kita. Toh, kita bisa saja membuat `list` yang tidak pernah bertambah/berkurang dan terdiri dari elemen dengan tipe yang sama. Sekarang kita akan melihat dua perbedaan besar lainnya, yang pertama berkaitan dengan cara kerja *slicing* (pemotongan) dan yang kedua berkaitan dengan bagaimana operasi perhitungan dipropagasikan ke berbagai elemen berbeda. Kita akan membahas masing-masing di dalam bagian tersendiri.

2 Pengindeksan dan Pemotongan (*Slicing*)

Pengindeksan untuk larik bekerja dengan cara yang persis sama seperti pada `list`, yaitu menggunakan tanda kurung siku:

```
xs = np.arange(10,16)
print(xs)
```

Output:

```
[10 11 12 13 14 15]
```

```
xs[2]
```

Output:

```
12
```

Pada pandangan pertama, *slicing* untuk larik tampak identik dengan cara kerja *slicing* pada `list`:

```
xs = np.arange(10,16)
print(xs[2:4])
```

Output:

```
[12 13]
```

Kita dapat terus menambahkan contoh lain, misal `xs[-1]` mengakses elemen terakhir dalam larik, `xs[1:5:2]` menggunakan ukuran langkah (*stride*), dan seterusnya. Namun, terdapat perbedaan yang krusial, yaitu bahwa potongan dari larik merupakan sebuah tampilan (*view*) dari larik aslinya:

```
r = np.array([11,7,19,22])
print(r)
```

Output:

```
[11  7 19 22]
```

```
sli = r[1:3]
print(sli)
```

Output:

```
[ 7 19]
```

```
sli[0] = 55
print(sli)
```

Output:

```
[55 19]
```

```
print(r)
```

Output:

```
[11 55 19 22]
```

Contoh ini persis sama dengan apa yang kita lakukan dalam tutorial Python menggunakan `list`. Seperti yang mungkin kita ingat dari sana, potongan (*slice*) pada `list` secara *de facto* menghasilkan salinan (*copy*), sehingga `list` aslinya tidak terpengaruh oleh perubahan selanjutnya pada *slice* tersebut. Sifat *slice* inilah yang mengarahkan kita untuk menggunakan `L[:]` guna mendapatkan salinan dari `list` `L`. Di sisi lain, larik NumPy dibuat efisien, menghilangkan kebutuhan untuk menyalin data. Tentu saja, kita semua harus selalu ingat bahwa beberapa *slice* yang berbeda semuanya tetap merujuk pada larik dasar yang sama. Dalam beberapa kasus, jika kita benar-benar memerlukan salinan nyata, kita dapat menggunakan `numpy.copy()`:

```
r = np.array([11,7,19,22])
sli = np.copy(r[1:3])
```

```
sli[0] = 55
print(sli)
```

Output:

```
[55 19]
```

```
print(r)
```

Output:

```
[11  7 19 22]
```

Sangat mudah dilihat bahwa kita juga bisa menyalin seluruh larik sekaligus, tanpa berdampak apa pun pada larik aslinya:

```
r2 = np.copy(r)
r2[1] = 0
print(r2)
```

Output:

```
[11  0 19 22]
```

```
print(r)
```

Output:

```
[11  7 19 22]
```

Dalam studi komputasi numerik, kita akan sering membuat salinan larik di dalam fungsi-fungsi, dengan spirit bahwa kita memengaruhi dunia eksternal hanya melalui nilai pemberian (*return value*) dari sebuah fungsi saja. Meskipun begitu, cara ini hanyalah pilihan pedagogis (pembelajaran). Pada praktiknya, kinerjanya akan jauh lebih baik jika kita memodifikasi suatu larik "langsung di tempat" (*in place*).

Perbedaan lain antara `list` dan array, yang akan diperluas pembahasannya pada tutorial-tutorial mendatang, berkaitan dengan sifat *broadcasting*. Secara kualitatif, *broadcasting* bermakna bahwa NumPy cukup tahu bagaimana menangani entitas yang dimensi-dimensinya tidak sepenuhnya cocok. Berikut adalah contohnya:

```
r = np.array([11, 7, 19, 22])
r[1:3] = 55
print(r)
```

Output:

```
[11 55 55 22]
```

Sebenarnya kode di atas adalah contoh serupa yang pernah kita coba di tutorial Python menggunakan `list`. Pada contoh tutorial tersebut, kita menerima `TypeError`, karena Python tidak tahu bagaimana menangani satu angka tunggal di sisi kanan, ketika sisi kirinya membutuhkan setidaknya dua angka (yaitu, Python mengharapkan adanya objek *iterable* di sisi kanannya). Sebaliknya, untuk kasus larik, sebuah nilai di sisi kanan disebarkan (*broadcasted*) ke sejumlah ruang (*slots*) yang diperlukan.

Karena kita baru saja membahas tentang *slicing* dan bagaimana sebuah nilai dapat di-*broadcast* ke beberapa slot sekaligus, di sini menjadi tempat yang tepat untuk melihat idiom larik NumPy yang sangat umum, yaitu "irisan segala" (*everything slice*) atau `xs[:]`. Ingat, karena *slice* dari larik adalah sekadar tampilan ke larik aslinya, `xs[:]` tidak dapat digunakan untuk membuat salinan larik. Akan tetapi, sintaksis ini cukup berguna seperti ditunjukkan di bawah.

```
xs = np.arange(10,16)
print(xs)
xs[:]  = 7
print(xs)
```

Output:

```
[10 11 12 13 14 15]
[7 7 7 7 7 7]
```

Kita menggunakan *everything-slice* untuk mem-*broadcast* angka 7 ke semua slot larik yang sudah ada sebelumnya.

Kita mungkin bertanya-tanya mengapa kita memerlukan *everything-slice*. Jawabannya seharusnya sudah jelas jika kita melihat bahwa:

```
xs = 7
print(xs)
```

Output:

```
7
```

Jika kita tidak menggunakan *everything-slice*, Python akan berpikir bahwa kita sedang membuat sebuah variabel integer baru yang kebetulan saja bernama `xs`, padahal itu (hampir bisa dipastikan) bukanlah niat kita sebenarnya.

Kita telah melihat beberapa sintaksis baru yang diizinkan oleh larik atau *array* dalam NumPy. Bagian selanjutnya akan membahas operasi larik semacam ini secara lebih terperinci dan menyoroti kekuatan-kekuatannya.

3 Vektorisasi

Saat memperkenalkan larik di atas, kita telah sebutkan bahwa larik memungkinkan kita untuk menghindari, setidaknya sebagian besar, keharusan menulis perulangan (*loops*). Sekarang saatnya untuk melihat apa maksudnya.

```
xs = np.arange(10,16)
ys = np.arange(20,26)
print(xs)
```

Output:

```
[10 11 12 13 14 15]
```

```
print(ys)
```

Output:

```
[20 21 22 23 24 25]
```

```
print(xs + ys)
```

Output:

```
[30 32 34 36 38 40]
```

Kita lihat bahwa untuk larik NumPy, penambahan (*addition*) diinterpretasikan sebagai operasi elemen demi elemen (*elementwise*), yakni setiap pasang elemen dijumlahkan secara berurutan. Kemampuan untuk melakukan operasi massal semacam ini pada seluruh elemen di dalam sebuah larik (atau dua larik) secara bersamaan sering kali disebut sebagai **vektorisasi**. Perhatikan betapa berbedanya operasi ini dari perilaku `list` Python:

```
xs = list(range(10,16))
ys = list(range(20,26))
```

```
xs
```

Output:

```
[10, 11, 12, 13, 14, 15]
```

```
ys
```

Output:

```
[20, 21, 22, 23, 24, 25]
```

```
xs+ys
```

Output:

```
[10, 11, 12, 13, 14, 15, 20, 21, 22, 23, 24, 25]
```

```
news = [x+y for x,y in zip(xs,ys)]
news
```

Output:

```
[30, 32, 34, 36, 38, 40]
```

Untuk tipe data `list`, tanda + berfungsi sebagai operator penggabungan (*concatenate*), alih-alih melakukan penambahan secara matematis per elemen. Jadi, untuk menghasilkan output yang sama dengan yang diberikan oleh larik menggunakan tanda + sederhana, pada kasus tipe data `list` kita harus menggunakan *`list` comprehension* dipadukan dengan `zip()`.

Kembali ke array NumPy, perkalian juga diinterpretasikan sebagai operasi elemen demi elemen (dan hal yang sama berlaku untuk pengurangan, pembagian, serta perpangkatan eksponen):

```
xs = np.arange(10,16)
ys = np.arange(20,26)
print(xs*ys)
```

Output:

```
[200 231 264 299 336 375]
```

```
np.sum(xs*ys)
```

Output:

```
1705
```

Di sini, kita sekaligus memperkenalkan `numpy.sum()`, yang menjumlahkan seluruh elemen dari sebuah array NumPy. Dengan demikian, baris terakhir tersebut secara sederhana sedang menghitung perkalian titik (*scalar product*) dari dua buah vektor, hanya dalam satu baris yang sangat ringkas.

Kita mungkin bertanya-tanya apakah kita bisa menggunakan fungsi `sum()` bawaan yang disediakan Python. Jawabannya adalah, meskipun kedua opsi tersebut sah saja untuk digunakan, dalam praktiknya fungsi bawaan NumPy bekerja jauh lebih cepat. Pelajaran penting di sini adalah kita harus selalu mendayagunakan fungsionalitas NumPy sebisa mungkin.

Sama menariknya dengan operasi *elementwise* antara dua array, adalah kemampuan NumPy untuk menggabungkan sebuah larik dengan skalar, yang merupakan konsekuensi dari sifat *broadcasting* yang telah disebutkan sebelumnya. NumPy tahu cara menafsirkan entitas dengan bentuk (*shapes*) yang saling berbeda namun kompatibel.

```
xs = np.arange(10,16)
print(xs)
```

Output:

```
[10 11 12 13 14 15]
```

```
print(2*xs)
```

Output:

```
[20 22 24 26 28 30]
```

Kita mungkin ingat bahwa perkalian `list` dengan sebuah skalar akan menghasilkan output yang sangat berbeda (sebenarnya, kita pernah menggunakan sifat tersebut pada bagian sebelumnya ketika kita menuliskan `[0.]*5`). Di sisi lain, NumPy tahu bagaimana cara menggabungkan skalar dengan sebuah larik untuk melakukan perkalian ke semua elemen (jadi ini adalah satu lagi kasus yang kita tidak memerlukan *`list` comprehension*). Kita dapat memikirkan apa yang terjadi di sini sebagai direntangkannya angka 2 menjadi sebuah larik yang memiliki ukuran sama dengan `xs`, lalu setelah itu perkalian *elementwise* dilakukan. Dengan begitu, tidak perlu diragukan lagi, kita juga bisa melakukan berbagai operasi lain pada larik NumPy dengan skalar.

```
xs = np.arange(10,16)
print(1/xs)
```

Output:

```
[0.1        0.09090909 0.08333333 0.07692308 0.07142857 0.06666667]
```

Kombinasi antara *broadcasting* (kita dapat melakukan operasi matematis antara skalar dengan larik) dan vektorisasi (kita hanya menuliskan sebuah ekspresi tetapi perhitungannya dieksekusi secara serentak ke semua elemen) bisa jadi agak sulit dipahami pada awalnya. Namun, fitur-fitur ini sesungguhnya sangat luar biasa tangguh (baik secara fleksibilitas ekspresi maupun efisiensi) begitu kita sudah terbiasa dengannya.

Selain tahu bagaimana cara mengeksekusi operasi *elementwise* antarlarik (maupun antara larik dan skalar), NumPy mencakup banyak fungsi yang terlihat mirip dengan padanan fungsi matematisnya. Misalnya, `numpy.sqrt()`, `numpy.exp()`, `numpy.log()`, `numpy.sin()` bekerja sesuai dengan ekspektasi kita. Perbedaan krusial dari fungsi matematika biasanya adalah bahwa fungsi-fungsi NumPy ini sudah dirancang untuk memproses larik utuh sehingga nyaris selalu bekerja lebih cepat. Fungsi-fungsi NumPy lain yang mungkin masih asing bagi kita, tetapi bisa sangat berguna, antara lain `numpy.isfinite()` atau `numpy.sort()`.

Perhatikan bahwa seluruh fungsi yang telah kita bahas sejauh ini bekerja dengan menerima input larik dan mengembalikan sebuah larik. NumPy juga memiliki fungsi yang menerima larik dan mengembalikan nilai skalar, seperti halnya `numpy.sum()` yang sudah kita temui di atas. Fungsi-fungsi ini diberi nama yang intuitif, misalnya `numpy.prod()` menghitung produk perkalian dari seluruh elemen dan `numpy.amin()` mengembalikan nilai elemen terkecil. Fungsi lainnya yang sangat membantu adalah `numpy.argmax()`, yang mengembalikan letak indeks dari nilai terbesarnya.

NumPy bahkan memiliki satu buah modul utuh bernama `numpy.random`, dengan ragam fungsi sangat berguna dan teroptimasi yang terkait dengan angka-angka acak (*random numbers*). Sangat direkomendasikan untuk meluangkan sedikit waktu guna membaca dokumentasi NumPy agar kita lebih familier dengan fungsi-fungsi semacam ini, sebab fungsi tersebut sangatlah intuitif dan efisien.

Kita juga dapat menggunakan fungsionalitas di dalam NumPy untuk meracik fungsi bebas buatan kita sendiri yang tahu cara memproses larik. Contohnya, pada kode berikut:

```
def fa(xs):
    return 1/np.sqrt(xs**2+1)
xs = np.arange(10,16)
print(fa(xs))
```

Output:

```
[0.09950372 0.09053575 0.08304548 0.0766965  0.07124705 0.06651901]
```

Kita manfaatkan `numpy.sqrt()`, yang sudah tahu cara menangani argumen larik, lalu operasi pemangkatan (*squaring*), dan operasi penambahan angka 1, masing-masing ditangani langsung oleh NumPy itu sendiri (alias, ditafsirkan sebagai sesuatu yang merujuk pada larik utuh juga).

Sekarang, kita siap untuk mengimplementasikan ulang suatu fungsi bernama

"`tots`". Alih-alih menuliskan kode dengan cara `list` berikut,

```
import math
def f(x):
    return 1/math.sqrt(x**2+1)
ws = list(range(30,36))
xs = list(range(10,16))
tots = [w*f(x) for w,x in zip(ws,xs)]
print(tots)
```

Output:

```
[2.9851115706299676, 2.8066081273180745, 2.657455355319679,
↪ 2.5309844631963223, 2.422399699588928, 2.3281653683320878]
```

solusi baru kita, yang sepenuhnya memakai larik, secara sederhana adalah:

```
def fa(xs):
    return 1/np.sqrt(xs**2+1)
ws = np.arange(30,36)
xs = np.arange(10,16)
tots = ws*fa(xs)
print(tots)
```

Output:

```
[2.98511157 2.80660813 2.65745536 2.53098446 2.4223997  2.32816537]
```

Perhatikan bagaimana baris:

```
tots = [w*f(x) for w,x in zip(ws,xs)]
```

yang merupakan suatu *list comprehension*, digantikan menjadi:

```
tots = ws*fa(xs)
```

Sekali lagi, begitu kita terbiasa dengan metode semacam ini, kita akan benar-benar memahami betapa praktisnya eksekusi operasi-operasi matematis menggunakan larik dari NumPy.

Studi Kasus

Kasus 1

Buatlah sebuah program Python dengan NumPy untuk melakukan operasi berikut pada sebuah larik satu dimensi $x = [2, 4, 6, 8, 10]$, yaitu: (1) Hitung jumlah seluruh elemen, nilai rata-rata elemen, nilai elemen terbesar,dan nilai elemen terkecil; (2) Bentuk larik baru yang setiap elemennya merupakan hasil

dari $y = 3x + 1$; dan (3) Tampilkan semua hasilnya.

Solusi

Kita bentuk terlebih dahulu larik satu dimensi x menggunakan `np.array`. Selanjutnya, beberapa besaran dasar dapat dihitung dengan fungsi-fungsi NumPy seperti `np.sum`, `np.mean`, `np.max`, dan `np.min`. Untuk membentuk larik baru y, kita cukup menuliskan ekspresi $3x + 1$ secara langsung, karena NumPy akan menerapkan operasi tersebut ke seluruh elemen larik.

```python
import numpy as np

# Membentuk larik satu dimensi
x = np.array([2, 4, 6, 8, 10])

# Menghitung beberapa besaran
jumlah = np.sum(x)
rata_rata = np.mean(x)
maksimum = np.max(x)
minimum = np.min(x)

# Operasi vektor pada seluruh elemen
y = 3 * x + 1

# Menampilkan hasil
print("x =", x)
print("Jumlah elemen =", jumlah)
print("Rata-rata =", rata_rata)
print("Elemen terbesar =", maksimum)
print("Elemen terkecil =", minimum)
print("y = 3x + 1 =", y)
```

Output:

```
x = [ 2  4  6  8 10]
Jumlah elemen = 30
Rata-rata = 6.0
Elemen terbesar = 10
Elemen terkecil = 2
y = 3x + 1 = [ 7 13 19 25 31]
```

Sekarang kita jelaskan hasilnya. Untuk larik

$$x = [2, 4, 6, 8, 10],$$

jumlah seluruh elemennya adalah

$$2 + 4 + 6 + 8 + 10 = 30.$$

Karena banyak elemen ada 5, rata-ratanya adalah

$$\frac{30}{5} = 6.$$

Elemen terbesar pada larik tersebut adalah

$$10,$$

sedangkan elemen terkecilnya adalah

$$2.$$

Selanjutnya, larik baru dibentuk dari hubungan

$$y = 3x + 1.$$

Artinya, setiap elemen pada x dikalikan 3, lalu ditambah 1. Dengan demikian diperoleh

$$\begin{aligned} y &= [3(2) + 1,\ 3(4) + 1,\ 3(6) + 1,\ 3(8) + 1,\ 3(10) + 1] \\ &= [7, 13, 19, 25, 31]. \end{aligned}$$

Kasus 2

Buatlah sebuah program Python dengan NumPy untuk menghasilkan 11 titik yang terdistribusi **merata** pada interval

$$0 \leq x \leq 5$$

menggunakan `numpy.linspace`. Hati-hati perhatikan batas tepat kiri maupun kanan pada inverval tersebut terkait dengan tanda kesamaan/ketaksamaan. Setelah itu:

1. hitung nilai fungsi $f(x) = x^2 - 2x + 3$ pada setiap titik tersebut,
2. tentukan indeks elemen tempat nilai fungsi terbesar berada,
3. tampilkan titik-titik x dan nilai $f(x)$.

Tuliskan programnya dan jelaskan hasilnya.

Solusi

Pada soal ini, kita gunakan `np.linspace(0, 5, 11)` untuk membentuk 11 titik yang berjarak sama dari 0 sampai 5. Setelah itu, fungsi

$$f(x) = x^2 - 2x + 3$$

dihitung pada semua elemen larik x sekaligus. Untuk menentukan letak nilai fungsi terbesar, kita gunakan `np.argmax`.

```python
import numpy as np

# Membentuk 11 titik dari 0 sampai 5
x = np.linspace(0, 5, 11)

# Menghitung nilai fungsi pada setiap titik
f = x**2 - 2*x + 3

# Menentukan indeks nilai fungsi terbesar
indeks_maks = np.argmax(f)

# Menampilkan hasil
print("x =", x)
print("f(x) =", f)
print("Indeks nilai fungsi terbesar =", indeks_maks)
print("Nilai fungsi terbesar =", f[indeks_maks])
print("Terjadi pada x =", x[indeks_maks])
```

Output:

```
x = [0.  0.5 1.  1.5 2.  2.5 3.  3.5 4.  4.5 5. ]
f(x) =
[ 3.    2.25  2.    2.25  3.    4.25  6.    8.25 11.   14.25 18.  ]
Indeks nilai fungsi terbesar = 10
Nilai fungsi terbesar = 18.0
Terjadi pada x = 5.0
```

Mari kita uraikan hasil tersebut. Perintah

`np.linspace(0, 5, 11)`

menghasilkan 11 titik yang tersebar merata dari 0 sampai dengan 5, yaitu

$$x = [0,\ 0.5,\ 1,\ 1.5,\ 2,\ 2.5,\ 3,\ 3.5,\ 4,\ 4.5,\ 5].$$

Karena ada 11 titik pada interval sepanjang 5, selisih antartitik adalah

$$\Delta x = \frac{5-0}{11-1} = 0.5.$$

Berikutnya kita hitung nilai fungsi

$$
\begin{aligned}
f(0) &= 3, \\
f(0.5) &= 0.25 - 1 + 3 = 2.25, \\
f(1) &= 1 - 2 + 3 = 2, \\
f(1.5) &= 2.25 - 3 + 3 = 2.25, \\
f(2) &= 4 - 4 + 3 = 3, \\
f(2.5) &= 6.25 - 5 + 3 = 4.25, \\
f(3) &= 9 - 6 + 3 = 6, \\
f(3.5) &= 12.25 - 7 + 3 = 8.25, \\
f(4) &= 16 - 8 + 3 = 11, \\
f(4.5) &= 20.25 - 9 + 3 = 14.25, \\
f(5) &= 25 - 10 + 3 = 18.
\end{aligned}
$$

Jadi larik nilai fungsi adalah

$$f = [3,\ 2.25,\ 2,\ 2.25,\ 3,\ 4.25,\ 6,\ 8.25,\ 11,\ 14.25,\ 18].$$

Nilai terbesar dari larik tersebut adalah 18 dan nilai ini terjadi saat $x = 5$. Karena NumPy menggunakan indeks mulai dari 0, posisi elemen terakhir pada larik f adalah indeks 10.